



MODULE :

Le Cloud Computing & DevOps

Pr. F. Benabbou
Master DSBD & IA
Faculté des Sciences Ben M'Sik Casablanca





TABLE OF CONTENTS

01 CLOUD COMPUTING

- Introduction générale
- La Virtualisation
- Les concepts de base du Cloud Computing
- Technologies émergentes du CC : Edge, Fog, ...
- Étude de cas et projet pratique

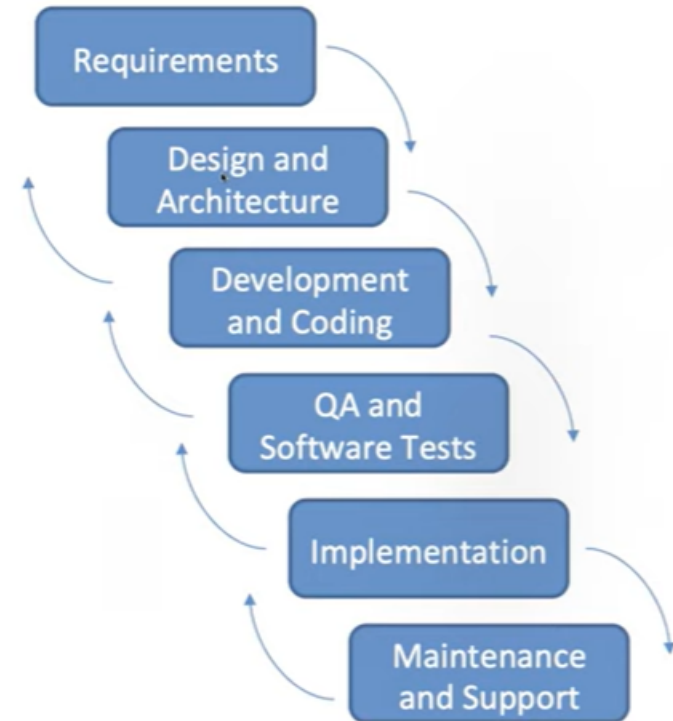
02 DevOps & Cloud

- La philosophie DeVops
- Version control systems (git)
- Integration Continuous
- Tests automatisés
- Deploiment Continu CD
- Infrastructure en tant que Code (IaC)
- Surveillance et Journalisation
- Étude de cas et projet pratique



Traditional software development Lifecycle (TSDL)

- Les méthodes de développement de logiciels classique sont souvent désignée par **Waterfall** or **Silos Model**.
- Cycle de développement
 - **Analyse** des besoins
 - **Conception** : Production d'une spécification de l'artefact logiciel
 - **Développement** : Développement du logiciel
 - **Test** : Examen de la qualité du produit ou du service, recherche et correction des défauts
 - **Implémentation** : Déploiement en production.
 - **Maintenance**





Limites des methods classiques

- **Cloisonnement (Silos) des équipes de développement et d'exploitation**
 - Ces équipes étaient séparées, avec une interaction limitée.
 - Les développeurs écrivaient le code et les services opérationnels le déployaient et le maintenaient
 - Problèmes de communication, des retards et des frictions entre les équipes.
- **Cycles de développement longs**
 - Le développement suivait un modèle en cascade, dans lequel les projets progressaient linéairement à travers des phases telles que la collecte des besoins, la conception, le développement, les tests et le déploiement.
 - Chaque phase devait être **achevée** avant de passer à la suivante, ce qui entraînait de **longs cycles** entre les versions.
- **Automatisation limitée**
 - Les tests étaient généralement effectués manuellement et le déploiement était souvent pris en charge par les opérations, avec peu de participation des développeurs.
 - Cela augmentait le risque d'erreurs lors du déploiement et rendait le processus plus lent.
- **Retour d'information tardif**
 - Les feedback interviennent tardivement dans le processus, parfois seulement après la sortie du produit.
 - Il était donc difficile de répondre rapidement aux besoins des clients ou de procéder à des

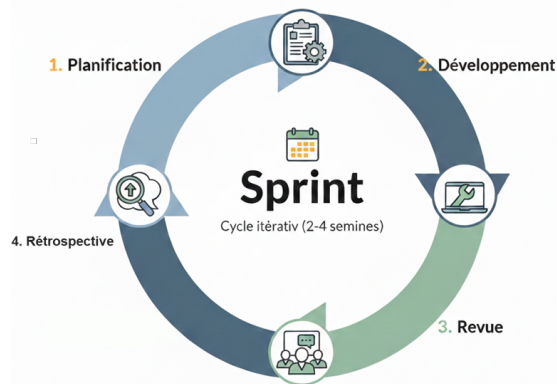
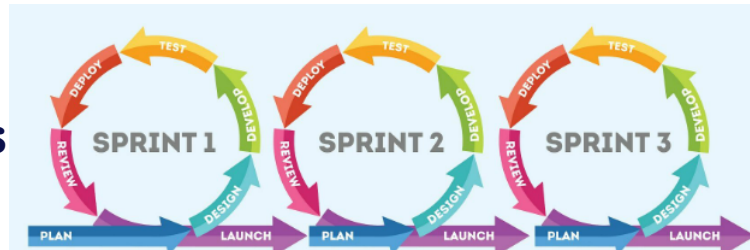


Les méthodes Agiles

- Échec du "Waterfall" (Années 80-90)
- L'émergence des méthodes isolées (1990-2000): Plusieurs méthodes voient le jour indépendamment.
 - ✓ 1995 : Jeff Sutherland et Ken Schwaber présentent le **Scrum** à une conférence.
 - ✓ 1996 : Kent Beck introduit **l'eXtreme Programming** (XP), qui met l'accent sur la qualité technique du code.
- Naissance officiel: Le Manifeste Agile (2001)
 - ✓ 17 experts du logiciel se réunissent dans une station de ski à Snowbird (Utah).
 - ✓ Ils constatent qu'ils travaillent tous différemment mais partagent les mêmes **valeurs**.
 - ✓ Ils rédigent le **Manifeste Agile**, définissant les **4 valeurs** et **12 principes** qui sont utilisés encore aujourd'hui.

Les méthodes Agiles

- L'Agilité est une approche itérative et incrémentale où on ne planifie pas tout le projet à l'avance, mais on avance par **petits pas**.
- Le projet est découpé en **cycles** appelés **Sprints**
- Principes clés
 - ✓ Développer en cycles **courts (sprints** , généralement de 1 à 4 semaines).
 - ✓ Collaboration avec le client
 - ✓ Développer et **s'adapter** rapidement aux **changements**
 - ✓ Livrer des versions fonctionnelles **fréquentes**





Comparaison MA vs TSDLC

Critère	Méthode Agile	Traditional SDLC (Waterfall)
Méthodologie	Itérative et incrémentale, avec des sprints courts (1-4 semaines)	Séquentielle, chaque phase dépend de la précédente
Flexibilité et Adaptation	Très flexible, les exigences peuvent évoluer au fil du projet	Moins flexible, les exigences sont définies en début et ne changent pas facilement
Planification et Documentation	Documentation légère, planification continue	Documentation détaillée et planification complète dès le début
Livraison et Feedback	Livraisons fréquentes et incrémentales, feedback continu	Livraison finale à la fin du projet, feedback tardif
Gestion des Risques	Risques identifiés tôt et ajustés rapidement	Risques identifiés tardivement, souvent après les tests
Rôle des Parties Prenantes	Participation active et continue des parties prenantes	Implication au début pour les exigences et à la fin pour la validation
Équipe de Développement	Autonomie des équipes, collaboration transversale	Équipes spécialisées par phase (analystes, développeurs, testeurs)
Adaptation à la Taille du Projet	Adapté aux projets de taille petite à moyenne avec exigences changeantes	Adapté aux projets de grande envergure avec exigences stables
Coûts et Temps	Peut avoir des coûts plus élevés si les priorités changent souvent, mais peut livrer plus rapidement	Moins coûteux au début, mais peut devenir plus cher si des erreurs sont découvertes tardivement



Les méthodes Agiles

- Une méthode agile est une méthode de **gestion de projet**, elle couvre principalement :
 - ✓ L'organisation de l'équipe en insistant sur la collaboration et communication
 - ✓ La planification des tâches en cycle court
 - ✓ Le suivi du projet en favorisant : la flexibilité face au l'amélioration continue
- Parmi les frameworks agiles les plus populaires figurent
 - ✓ Scrum,
 - ✓ Kanban,
 - ✓ Extreme Programming (XP)
 - ✓ Lean Software Development, etc.



- L'Agilité a permis aux développeurs de produire du code par cycles courts (Sprints):

Méthode Agile et devOps

Il a introduit la collaboration, le développement itératif et le retour d'information des clients.

- ✓ L'agilité répond à la question :Comment développer **plus vite** et **mieux** ?
- ✓ L'Agilité s'arrêtait souvent à la livraison du code: Plan → Design → Build → Test
- Le DevOps étend ce cycle jusqu'à l'utilisateur final : ... → Release → Deploy → Operate → Monitor.
- On ne s'arrête plus à "le code est fini", on va jusqu'à "le code est en ligne et fonctionne".
- Le DevOps n'est pas une alternative à l'Agilité, mais son extension au cycle de vie opérationnel.
- Il transpose les concepts de **développement logiciel** à la **gestion d'infrastructure**.



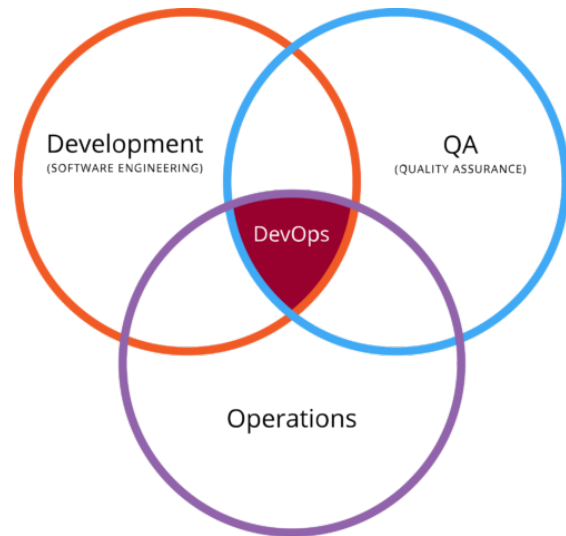
Mouvement DevOps

- Le mouvement DevOps a vu le jour entre 2007 et 2008 et a été inventé par **Patrick Debois**, qui a organisé la première conférence **DevOpsDays** en Belgique en 2009.
- Les communautés des opérations informatiques et du développement de logiciels ont exprimé les limites du modèle traditionnel de développement de logiciels.
- L'objectif principal était :
 - ✓ de combler le fossé entre les **deux équipes** traditionnellement **cloisonnées: développement et l'exploitation**
 - ✓ d'améliorer la **collaboration**
 - ✓ et **d'accélérer** la livraison des logiciels sans compromettre la qualité.

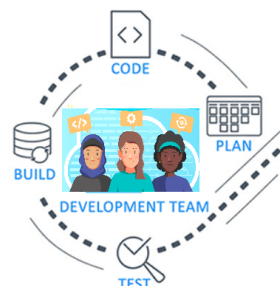


C'est quoi DevOps?

- Le DevOps est une méthode de développement de logiciels qui combine le développement de logiciels (Dev) et les opérations informatiques (Ops).
- DevOps intègre toutes les fonctions de développement de logiciels, **du développement à l'exploitation**, au sein d'un **même cycle**.
- Il nécessite un niveau élevé de coordination entre les différentes parties prenantes du processus de développement de logiciels, à savoir le **développement**, l'assurance **qualité** et les **opérations**.

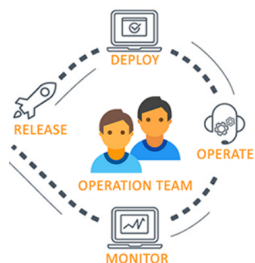


Parties prenantes



Le **développeur** est le créateur du code et le garant de sa qualité.

- Écrit le code
- Réalise les tests unitaires
- Participe au déploiement (configuration de *déploiement* et les critères de *surveillance*).



Le **Ops** (Opérations) est responsable de *l'exploitation* à la fois des **applications** et de **l'infrastructure** (serveurs, cloud, conteneurs)

- Automatise les déploiements
- Surveille le système (monitoring, alertes)
- Assure la disponibilité et la performance

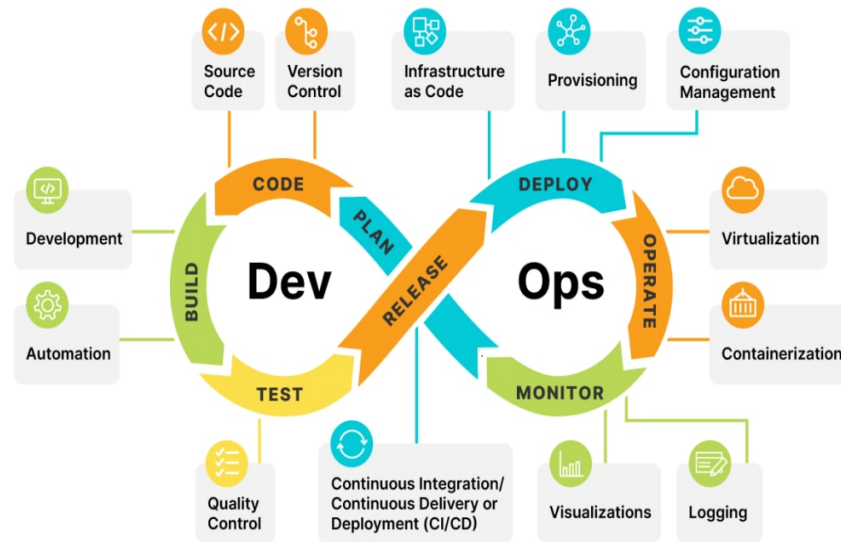


Le **QA (Quality Assurance)** assure la qualité des logiciels.

- Vérifie la qualité du logiciel
- Conçoit et exécute les tests (fonctionnels, automatisés, performance)
- Valide que les exigences sont respectées
- Participe à l'amélioration continue **garantir** des versions **fiables** et de **haute qualité**.

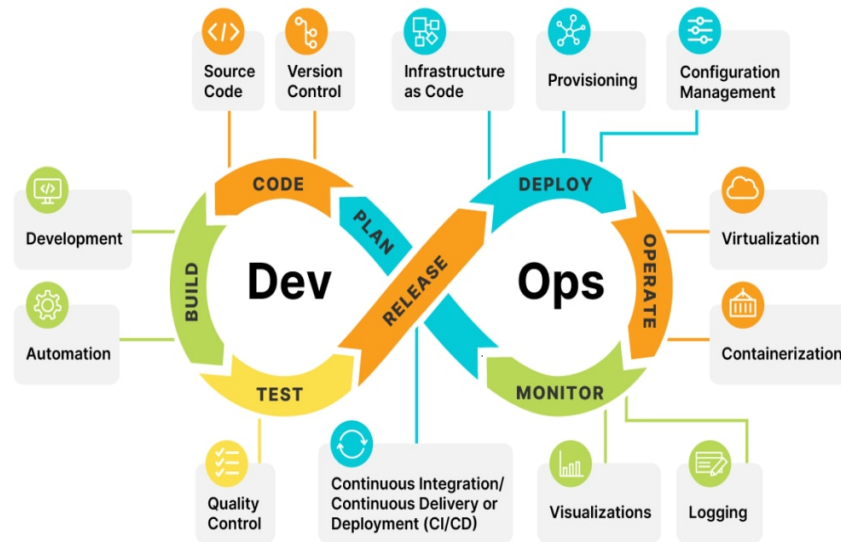
Cycle DevOps

- DevOps est un **cycle continu** qui couvre toute la vie d'une application, depuis l'idée jusqu'à la surveillance en production.
- La boucle DEV
 - ✓ Plan : Définition des besoins et classer par ordre de priorité.
 - ✓ Code : Écriture du code et gestion des versions dans le Dépôt (Git).
 - ✓ Build : Transformation du code en application exécutable : un artefact immuable (Regrouper les applications et les dépendances, Image Docker).
 - ✓ Test : Validation par la QA (Tests unitaires et intégration).



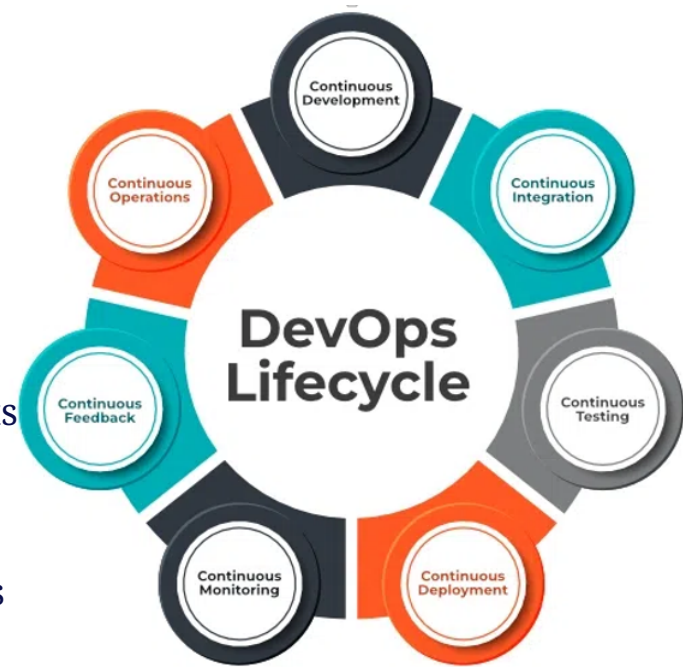
Cycle DevOps

- La boucle OPS (Déployer & Piloter)
 - ✓ Release : Validation finale de l'artefact.
 - ✓ Deploy : Le déploiement fait référence à la mise en production des applications dans les environnements de production, ce qui implique de gérer et configurer l'infrastructure nécessaire pour soutenir les applications.
 - ✓ Operate (Exploiter) : assurer le bon fonctionnement, gérer les serveurs, la performance et la disponibilité.
 - ✓ Monitor : Surveillance continue pour renvoyer du feedback au Plan (Collecter des métriques, Détecter les erreurs, Analyser les logs, régressions de performance, consommation anormale de ressources, Recevoir des alertes).



Le concept du "Continuous Everything"

- Le "Continuous Everything" est l'**automatisation** du cycle de vie DevOps : chaque phase du cycle de vie est automatisée pour se déclencher dès que la précédente produit un résultat.
 - ✓ **Développement & Intégration Continue (CI)** : Automatise les phases Code et Build, Fusion fréquente du code et validation automatisée pour détecter les conflits au plus tôt.
 - ✓ **Tests Continus** : Exécution systématique de suites de tests à chaque étape pour garantir la non-régression et la sécurité.
 - ✓ **Déploiement Continu (CD)** : Automatise les phases Release et Deploy pour livrer de la valeur aux utilisateurs de façon fluide et rapide.
 - ✓ **Opérations & Surveillance Continues** : Monitoring en temps réel des performances et de la santé des systèmes pour anticiper les incidents.
 - ✓ **Feedback Continu** : Collecte immédiate des retours utilisateurs et techniques pour alimenter le prochain cycle





Les 5 piliers CALMS DevOps

- John Willis et Damon Edwards ont inventé l'acronyme CAMS en 2010, plus tard étendu à CALMS par Jez Humble.
- CALMS signifie :
 - ✓ Culture
 - ✓ Automation
 - ✓ Lean
 - ✓ Measurement
 - ✓ Sharing
- Chaque concept contribue au succès de la philosophie DevOps.



Les 5 piliers CALMS DevOps : culture

■ Culture

- ✓ C'est le plus important !!
- ✓ le changement culturel peut être considéré comme le cœur de toute transformation DevOps.
- ✓ La collaboration du Dev., Ops. QA est un élément fondamental de la culture DevOps
- ✓ Il faut instaurer un climat de confiance, de transparence et de "blameless culture" : « on ne cherche pas un coupable en cas d'erreur, mais une solution ».



Les 5 piliers CALMS DevOps: Automatisation

■ L'automatisation

- ✓ But : Remplacer les interventions humaines répétitives, **lentes et sujettes à l'erreur** par des processus pilotés par des scripts et des outils logiciels.
- ✓ L'idée de base est d'automatiser tout ce qui peut l'être (Build (CI), Tests, Déploiement (CD, Provisionnement infrastructure, Monitoring...) pour réduire l'erreur humaine et gagner en vitesse.
- ✓ Cela rend les productions fiables et sécurisées et garantit que chaque déploiement est identique et prévisible.
- ✓ Un autre bénéfice essentiel de l'automatisation c'est d'accélérer considérablement le time-to-market, tout en libérant les équipes pour qu'elles puissent se concentrer sur l'innovation et les activités à forte valeur ajoutée.



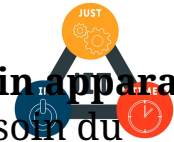
Les 5 piliers CALMS DevOps : Automatisation

■ L'automatisation

- ✓ But : Remplacer les interventions humaines répétitives, **lentes et sujettes à l'erreur** par des processus pilotés par des scripts et des outils logiciels.
- ✓ L'idée de base est d'automatiser tout ce qui peut l'être (tests, déploiements, infrastructure) pour réduire l'erreur humaine et augmenter la productivité de l'équipe.
- ✓ Cela rend les productions fiables, sécurisées, scalables et garantit que chaque déploiement est identique et prévisible.
- ✓ Un autre bénéfice essentiel de l'automatisation c'est d'accélérer considérablement le time-to-market, tout en libérant les équipes pour qu'elles puissent se concentrer plus sur le code pour développer de nouvelles fonctionnalités, et sur l'innovation et les activités à forte valeur ajoutée.
- ✓ L'automatisation est un élément clé d'un pipeline Continuous Integration et Continuous deployment (CI/CD).

Les 5 piliers CALMS DevOps: LEAN

- Le Lean cherche à maximiser la valeur client et réduire le délai en supprimant les tâches inutiles (le gaspillage ou "Muda").
- YAGNI (You Ain't Gonna Need It) : "Tu n'en auras pas besoin »**
 - ✓ C'est un principe issu de l'Extreme Programming (XP)
 - ✓ Ne développe jamais une fonctionnalité "**au cas où**" on en aurait besoin dans le futur
 - ✓ On se concentre sur ce qui est nécessaire aujourd'hui.
 - ✓ Écrire du code inutile est un gaspillage de **temps, d'énergie** et cela complexifie la maintenance
- JIT (Just-In-Time) : Le "Juste à Temps" numérique**
 - ✓ Origine industrielle : Inspiré de Toyota, *on ne produit une pièce que lorsqu'une commande est passée*. Cela élimine les stocks inutiles et les coûts d'entreposage.
 - ✓ Application au DevOps : C'est le principe du **déploiement à la demande**.
 - On ne prépare plus de serveurs des mois à l'avance "au cas où".
 - Les environnements sont créés et le code déployé **au moment exact où le besoin apparaît**.
 - Moins de ressources dormantes (coûts réduits) et une réponse immédiate au besoin du client.





Les 5 piliers CALMS DevOps : Measurement

■ Measurement

- ✓ Le principe est simple : on ne peut pas améliorer ce que l'on ne **mesure** pas.
- ✓ Les métriques fournissent des données objectives qui permettent de suivre les performances du système et du processus, identifier les points à améliorer et prendre des décisions basées sur des données (data-driven), et non pas sur des impressions.
- ✓ les 4 métriques **DORA** (DevOps Research and Assessment) sont les plus connues, et sont une référence pour évaluer la performance des équipes DevOps.
 - **Deployment Frequency** (Vitesse) : Mesure la **fréquence** à laquelle une application est **déployée** en production.
 - Indique la capacité à livrer rapidement des fonctionnalités
 - Montre le niveau d'automatisation et de maturité pipeline DevOps

Les 5 piliers CALMS DevOps : Measurement

■ Measurement

- ✓ les 4 métriques DORA (DevOps Research and Assessment).
 - **Lead Time for Changes** (Agilité): Mesure le **temps** entre la **validation** du code et son **déploiement** en production.
 - Indique la rapidité du pipeline
 - Permet de détecter les goulots d'étranglement
 - **Change Failure Rate** (Qualité) : Mesure le **pourcentage** de **déploiements** qui causent un **incident** ou nécessitent un correctif.
 - Indique la qualité des tests et du processus
 - Mesure la fiabilité des releases



Les 5 piliers CALMS DevOps : Measurement

■ Measurement

- ✓ les 4 métriques DORA (DevOps Research and Assessment).
 - **Time to Restore Service** (Résilience) : Mesure le **temps** moyen nécessaire pour **restaurer** un service après une panne.
 - Indique la capacité à réagir aux incidents
 - Montre l'efficacité du monitoring et des procédures
 - **Exemple:**
 - Deployment Frequency : Une fois / mois → faible maturité, Plusieurs fois / jour → forte maturité DevOps.
 - Lead Time for Changes: 2 semaines → pipeline lent, quelques heures → pipeline efficace.
 - Change Failure Rate : 30% d'échec → problème de qualité ou de tests 5% → bon niveau de fiabilité.
 - Time to Restore Service: 5 heures → réaction lente, 20 minutes → bonne résilience.



Les 5 piliers CALMS DevOps : Measurement

■ Measurement

- ✓ D'autres indicateurs Techniques :
 - Latence ou Temps de réponse,
 - Temps de build
 - Couverture de tests
 - Disponibilité du service
 - Satisfaction utilisateur



Les 5 piliers CALMS DevOps : Sharing

■ Sharing

- ✓ Le sharing consiste à favoriser le partage des connaissances et des meilleures pratiques acquises tout au long du processus car il améliore le travail collaboratif et éviter la dépendance à des individus spécifiques.
- ✓ Bonnes pratiques
 - *Documentations accessible à tous* : Procédures de déploiement, Architecture, système, configuration des serveurs, Runbooks (procédures en cas d'incident), guides CI/CD...
 - *La Source Unique de Vérité* (SSOT - Single Source of Truth) : Tout ce qui est nécessaire pour construire, déployer et comprendre le système doit être dans un système de contrôle de version (**Git**).
 - *Post-mortems sans blâme* : L'erreur est vue comme une défaillance du système, il faut analyser l'incident sans chercher un coupable, pour comprendre et améliorer le système.
 - Les *métriques* de performance doivent être publiques par défaut au sein de l'entreprise.

Les 3 Voies : Le standard de communication DevOps

■ Flow

- Les équipes dev. Et ops. (développement et d'exploitation) communiquent, partagent les informations et collabore tout au long du cycle de développement et de déploiement
- le flux de valeur doivent être : rapide, continu, sans blocage
- Ce qui permet d'avoir une livraison rapide, automatisation et de petits changements fréquents.

■ Feedback

- La deuxième voie, le Feedback, représente les boucles de rétroaction
- L'Ops informe immédiatement le Dev si la performance chute après un déploiement ou s'il y a des erreurs,...

■ Learning

- Ce qui permet au Dev de corriger en continue et rapidement
- La troisième voie de DevOps encourage une culture **d'expérimentation** et **d'apprentissage** pour s'améliorer rapidement
 - o il s'agit de prendre des risques, d'essayer de nouvelles idées et d'apprendre de ses échecs.
 - o apprendre efficacement de ses erreurs et monter en compétence .

